“✅ Done! Everything works.” Does it, though?

Every AI agent finishes this way — the same model that did the work, grading its own homework. shotlist is the open-source CLI your agent runs **outside the model**: real screenshots, drift-checked in CI. Receipts, not vibes.

THE CLAIM

`“I tested everything and it all works!”`

generated inside the model · no evidence · you find out in prod

vs

THE PROOF

`01-dashboard.png`

a real screenshot from a real browser · captured by `shotlist run` · drift-checked in CI

One committed YAML → every screenshot, regenerated

`shotlist run` boots your app, waits until it's *actually* ready, routes every shot to the right backend — and tears it all down afterwards.

`.shotlist.yaml` — committed with your code

```yaml
output:  dir: docs/screenshots
         readme: README.md
app:     command: "npm run dev"
         ready: {url: :5173}
shots:
 - {name: dashboard, kind: web}
 - {name: cli-help,  kind: cli}
 - {name: onboard, kind: session}
```

1 · validate

schema-checked — typos become one clear error line

2 · boot app + ready probe

waits on `url / port / log line` — never shoots a half-booted app

3 · route each shot by kind

one deterministic engine, four capture paths

`kind: web`

Playwright · Chromium — click / fill / wait steps first, flaky regions masked, animations off

`kind: cli · style: native`

your **real Terminal.app window** — your font, your theme (macOS)

`kind: cli · style: rendered`

PTY → ANSI→HTML → styled terminal card — any OS, CI-safe

`kind: session`

one persistent shell, one shot per step — env, cwd & background processes carry over

every run writes

- `01-*.png` numbered baseline shots
- `index.html` shareable proof gallery
- `README.md` <img> tags auto-spliced
- `manifest.json` machine-readable, for CI

4 · teardown — always

the app's process group is killed even on crash or Ctrl-C — no orphaned dev servers. `--keep-going` finishes partial runs; `retries: N` re-attempts flaky shots.

Real use case — README screenshots that never go stale: after every UI change, one `shotlist run` regenerates the entire set — same config, same app state, same pixels.

The agent authors once. Capture costs zero tokens, forever.

An AI assistant's only job is to *write* the config by reading your repo. Capture itself is a plain deterministic program — no model in the loop.

PHASE 1 · ONCE — AUTHORED, NOT EXECUTED, BY AI

any agent — or no agent

- Claude Code · /shotlist skill
- Cursor · rules file
- Codex · AGENTS.md
- a human · shotlist init

reads your repo

find the routes worth shooting, the dev-server command, the port, the CLI entry points

writes `.shotlist.yaml` · runs `shotlist validate`

plain YAML — reviewable in the PR like any other code

committed

the shot list now belongs to the repo, not to the agent that wrote it

PHASE 2 · EVERY TIME AFTER — DETERMINISTIC CLI

`$ shotlist run`

boots the app → captures every shot → tears it all down. Same pixels on your laptop, in CI, or from any agent's shell.

no model · no tokens · no cloud

fast, free, re-runnable — the agent is out of the loop the moment the YAML is committed

PR with real screenshot proof

"here's the feature working" — pixels, not claims

`docs/test-evidence.md`

one captioned shot per feature, spliced automatically

`index.html` proof gallery

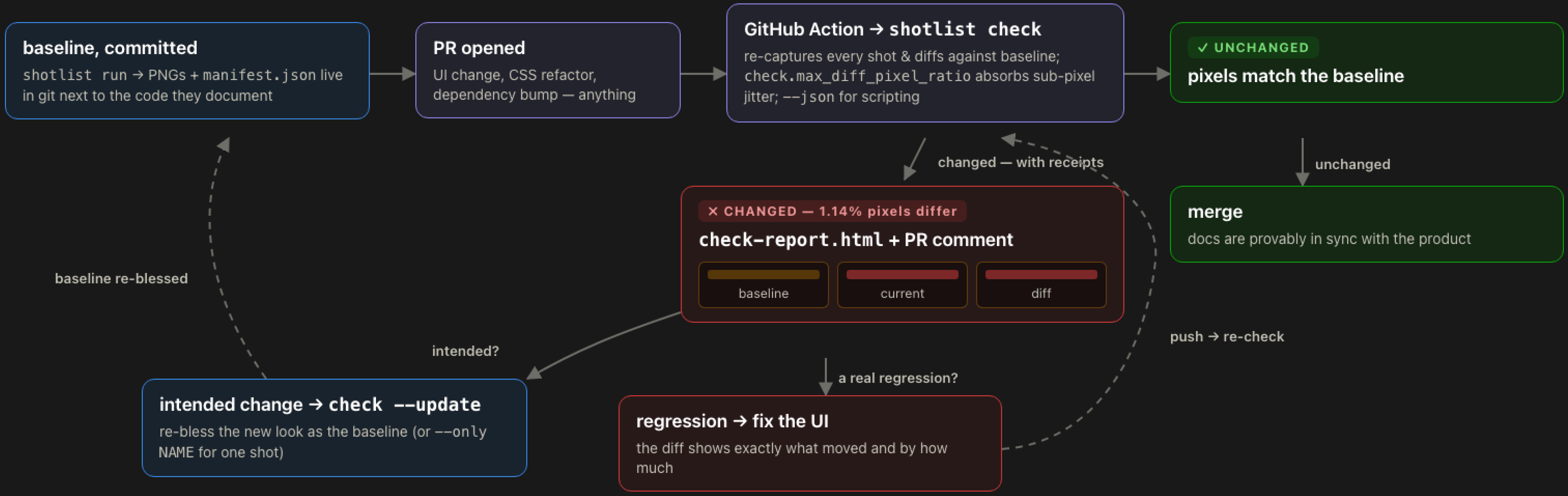
self-contained — open it, share it, attach it to the ticket

Real use case — your coding agent finishes a feature and attaches *actual screenshots* of it working to the PR — instead of a paragraph claiming it does.

SHOTLIST IN CI — THE DRIFT GATE

Screenshots that fail the build when they go stale

On every PR, `shotlist check` re-captures each shot and compares it to the committed baseline — drift blocks the merge, with receipts.



Real use case — a CSS refactor nudged a status card off-grid. CI flagged **1.14% pixel drift** with a baseline·current·diff report on the PR — caught before any user (or README reader) saw it.

SHOTLIST SESSIONS — STATEFUL FLOWS

Screenshot a flow, not a moment

A `session` runs every step in **one persistent shell** — env vars, cwd and background processes carry across steps, with one PNG per step.

KIND: SESSION — ONE PERSISTENT SHELL

STEP 1

`$ export API_KEY=demo-123`

set up state for everything that follows

`01-setup.png`
captured after the step

STEP 2 · LONG-RUNNING

`$ npm run dev &`
`# wait_ms: 3000 — let it boot`

backgrounded — it keeps running for later steps

`02-server.png`
dev server live, in-terminal

STEP 3 · SEES STEPS 1–2

`$ curl :5173/health → ok`

same shell: the key from step 1, the server from step 2 — still there

`03-health.png`
proof the flow works end-to-end

teardown

session closes → the backgrounded server is killed. Nothing orphaned — even on crash.

state carries over: environment variables · working directory · background processes — exactly like a human driving one terminal

style: rendered — terminal cards from a PTY, any OS, CI-safe

style: native — your real Terminal.app window on macOS

scrub: regex out timestamps & PIDs → byte-stable baselines

Real use case — a getting-started guide: install → boot → verify, one image per step, each later step depending on the earlier ones — regenerated in full on every release.

SHOTLIST · GET STARTED

Make your docs prove themselves.

1	`pip install shotlist`	plus a one-time `playwright install chromium`
2	`shotlist init`	scaffolds `.shotlist.yaml` — or let your coding agent write it from your repo
3	`shotlist run`	boots the app, captures every shot, splices the README, tears it all down

EVERY RUN WRITES

- `01-*.png` reproducible baseline screenshots
- `index.html` a shareable proof gallery
- `README.md` images spliced in automatically
- `shotlist check` the CI drift gate, with visual diffs

> **Dogfood or it didn't happen:** every slide in this deck was captured by shotlist itself — one `.shotlist.yaml` pointed at six local HTML files. Python · MIT · no cloud · no paid services.

⭐ `github.com/varmabudharaju/shotlist`

star it if your screenshots folder is a crime scene